Reviewer Pack — Quantum Deformation of Boolean Circuit Depth

Entry 708bb68681ed · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 20:03:23 UTC

Quantum Deformation of Boolean Circuit Depth

Entry ID: `708bb68681ed` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-30 20:03:23 UTC

1. Conjecture

Field A (mathematical branch): Quantum Deformations in Algebraic Geometry **Field B** (complexity object): Boolean Circuit Complexity

Statement:

> The quantum deformation degree of the algebraic variety associated with a boolean circuit C is upper bounded by the depth of C, i.e., QDD(C) ≤ D(C), where QDD(C) is the quantum deformation degree and D(C) is the depth of C.

Rationale (proposer's reasoning):

> Quantum deformations have been studied in algebraic geometry, and they often lead to rich structures. Applying these deformations to boolean circuits could potentially reveal hidden connections between the geometric properties of circuits and their computational complexity.

Taxonomy category: `QuantumDeformation` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `a2d485302d95767c`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Quantum deformation degree QDD(V_C) is within 3 units of depth D(C) across all 30 seeds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2

4. Novelty audit

Verdict: `ADJACENT_OK` against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `quantum deformation algebraic geometry boolean circuit complexity` - `QDD Boolean Circuit Depth relationship` - `algebraic variety quantum deformation upper bound boolean circuit depth`

Top relevant hits considered: - [http://arxiv.org/abs/2407.04826v1] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits - [http://arxiv.org/abs/2102.01659v2] Capacity and quantum geometry of parametrized quantum circuits - [http://arxiv.org/abs/1904.05483v2] Parallels Between Phase Transitions and Circuit Complexity? - [http://arxiv.org/abs/1712.05384v2] Simulation of low-depth quantum circuits as complex undirected graphical models - [http://arxiv.org/abs/2512.15901v1 Closed-Form Optimal Quantum Circuits for Single-Query Identification of Boolean Functions - [http://arxiv.org/abs/1207.6655v2] A 2D Nearest-Neighbor Quantum Architecture for Factoring in Polylogarithmic Depth

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.1s

5.1 Generated Python source

```python
import random

def generate_circuit(depth):
    if depth == 1:
        return [random.choice([0, 1])]
    else:
        subcircuits = [generate_circuit(random.randint(1, depth-1)) for _ in range(2)]
        return [random.choice([0, 1])] + subcircuits

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_max = 40
    instances_tested = 30
    metric_values = []
    conjecture_holds = True
    counterexample = ""

    for n in range(5, n_max + 1):
        circuit = generate_circuit(n)
        depth = len(circuit) - 1

        # Construct the algebraic variety V_C (simplified representation)
        # For simplicity, we use a polynomial representation of the circuit
        polynomial = [0] * (n + 1)
        for gate in circuit:
            if gate == 0:
                polynomial[0] += 1
            else:
```

```
                polynomial[-1] -= 1

        # Measure QDD(V_C) (simplified calculation)
        qdd = abs(sum(polynomial))

        metric_values.append(qdd / depth)

    mean_metric_value = sum(metric_values) / len(metric_values)
    std_metric_value = (sum((x - mean_metric_value) ** 2 for x in metric_values) / len(metric_valu

    if any(x > y + 3 for x, y in zip(metric_values, [depth for _ in range(len(metric_values))])):
        conjecture_holds = False
        counterexample = "QDD(V_C) is more than 3 units larger than D(C)"

    return {
        "metric_name": "Quantum Deformation Degree / Depth Ratio",
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = (sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(r
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value:.2f} std={std_metric_value:.2f} support_
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"QDD(V_C) > D(C) + 3\" first_failing_seed={first
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
s_tested': 30, 'n_max': 40, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'Quantum Deformation Degree / Depth Ratio', 'metric_value': 0.8888888888888888
TRIAL: {'metric_name': 'Quantum Deformation Degree / Depth Ratio', 'metric_value': 0.9444444444444444
TRIAL: {'metric_name': 'Quantum Deformation Degree / Depth Ratio', 'metric_value': 1.0, 'instances_te
TRIAL: {'metric_name': 'Quantum Deformation Degree / Depth Ratio', 'metric_value': 0.8333333333333334
```

```
TRIAL: {'metric_name': 'Quantum Deformation Degree / Depth Ratio', 'metric_value': 0.9444444444444444
TRIAL: {'metric_name': 'Quantum Deformation Degree / Depth Ratio', 'metric_value': 0.8888888888888888
TRIAL: {'metric_name': 'Quantum Deformation Degree / Depth Ratio', 'metric_value': 1.0277777777777777
TRIAL: {'metric_name': 'Quantum Deformation Degree / Depth Ratio', 'metric_value': 1.0555555555555556
TRIAL: {'metric_name': 'Quantum Deformation Degree / Depth Ratio', 'metric_value': 1.1944444444444444
TRIAL: {'metric_name': 'Quantum Deformation Degree / Depth Ratio', 'metric_value': 1.0277777777777777
RESULT: SUPPORTED mean=0.99 std=0.10 support_fraction=1.00
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> The test only considers circuits of depth up to 40, which may not be sufficient to capture the behavior of more complex circuits. The metric used is a simplified calculation of 'Quantum Deformation Degree' that does not accurately reflect the true quantum deformation degree of the algebraic variety associated with the circuit.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Safety rail: critic_challenge | original judge: The test results indicate that the conjecture holds true for all tested instances with a mean value of 0.99 and standard deviation of 0.10, meeting th | next: Further investigation is needed to confirm the conjecture's validity for circuits beyond depth 40 and to refine the metric used to calculate the 'Quantum Deformation Degree'.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	19638
2	preregistration	ollama_remote	glm4:latest	0	0	16147
3	novelty	ollama_remote	glm4:latest	0	0	11560
4	novelty	ollama_remote	glm4:latest	0	0	12495
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14821
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18124
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11845
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15390
9	critic	ollama_remote	glm4:latest	0	0	11035
10	judge	ollama_remote	glm4:latest	0	0	15714

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 146770 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/708bb68681ed.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71

2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/708bb68681ed.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/708bb68681ed.tar.gz` (if generated)